

大模型应用开发与 Agent 开发学习路线

0. 总体认知

大模型应用开发的核心，不是单纯“会调 API”，而是围绕 LLM 的能力边界构建一套工程系统。

LLM 的天然限制包括：

- 训练知识有限，无法天然知道私有数据和最新信息。
- 只能生成文本，不能直接访问外部世界或执行动作。
- 容易幻觉，尤其在缺少依据或上下文冲突时。
- 上下文窗口有限，长任务中容易遗忘、跑偏或被噪声干扰。
- 多步任务中缺少稳定的执行、验证、纠错和权限治理能力。

对应的工程补强路线是：

```
知识不足 -> RAG
不能行动 -> Tool / Function Calling
流程复杂 -> Agent
上下文有限 -> Memory / Context Engineering
不够稳定 -> Eval / Guardrails / Harness Engineering
工具生态复杂 -> MCP / Skills / CLI
多系统协作 -> Multi-Agent / A2A
```

一句话总结：

RAG 让模型有知识，Tool 让模型有行动能力，Agent 让模型能规划执行，Memory 让模型能延续上下文，Eval 和 Harness 让系统可靠、可控、可上线。

1. LLM 基础层

1.1 Transformer 基础

需要掌握：

- Tokenization：文本如何切成 token。
- Embedding：token 如何变成向量。
- Position Encoding / RoPE：为什么模型需要位置信息。
- Self-Attention：Q、K、V 如何计算词与词之间的关系。
- Multi-Head Attention：多个注意力头如何捕捉不同语义关系。
- Feed Forward Network：注意力之后的非线性变换。
- Residual Connection + LayerNorm：深层网络稳定训练的关键结构。
- Decoder-only 架构：GPT 类模型为什么适合自回归生成。

重点理解：

输入文本 -> token -> embedding + position -> 多层 attention/FFN -> logits -> softmax -> 下一个 token

Transformer 相比 RNN 和 CNN 的优势：

- 相比 RNN：可以并行计算，缓解长距离依赖衰减。
- 相比 CNN：不局限于局部窗口，可以直接建模全局 token 关系。

1.2 生成机制

需要掌握：

- logits
- softmax
- temperature
- top-p / nucleus sampling
- top-k
- repetition penalty
- max tokens
- stop sequence

参数经验：

- 代码生成、结构化抽取：低 temperature。
- 事实问答：低 temperature + RAG。
- 创意写作：适当提高 temperature。
- 严格 JSON 输出：低 temperature + JSON Schema / Structured Outputs。

1.3 幻觉与能力边界

LLM 幻觉的常见原因：

- 参数知识过期或不存在。
- 用户问题缺少必要信息。
- prompt 要求模型必须回答。
- 检索上下文质量低。
- 多文档信息冲突。
- 工具结果被误读。

解决方向：

- RAG 引入外部知识。
- Tool Calling 获取实时数据。
- 引用来源和证据约束。
- 不知道时允许拒答。
- 结构化输出校验。
- 评估集和回归测试。

2. Prompt 与结构化输出层

2.1 Prompt Engineering

基础能力：

- system / developer / user message 的职责划分。
- 任务目标、约束、输出格式、边界条件的清晰表达。
- few-shot 示例。
- 分步骤任务拆解。
- 输入内容与指令内容隔离，防止 prompt injection。

常见结构：

角色：你是什么
任务：你要做什么
输入：用户提供了什么
约束：不能做什么，必须遵守什么
输出：用什么格式返回
评价标准：什么算好答案

2.2 CoT 与推理方法

需要理解：

- CoT：让模型展开中间推理步骤。
- Zero-shot CoT：通过简单提示引导模型逐步思考。
- Few-shot CoT：提供问题、推理过程和答案示例。
- Self-Consistency：生成多条推理链再投票。
- Tree of Thoughts：探索多个候选分支。
- Graph of Thoughts：允许分支合并、回环和复用。

工程注意：

生产系统里不一定要把完整推理链暴露给用户，更常见的是让模型内部推理，最终输出结论、依据、引用和可验证步骤。

2.3 结构化输出

结构化输出是大模型应用开发的基础能力之一。

需要掌握：

- JSON mode
- JSON Schema
- Structured Outputs
- Function Calling 参数 schema
- pydantic / zod 校验
- 输出解析失败后的重试策略
- schema 版本演进

典型流程：

定义 schema -> 调用模型 -> 解析 JSON -> 校验字段 -> 失败重试 -> 进入业务逻辑

适合项目：

- 简历信息抽取。
- 合同条款抽取。
- 商品属性抽取。
- 发票、报告、工单结构化。

3. RAG 知识增强层

RAG 的本质是把模型的参数知识和外部知识库结合起来。经典 RAG 论文将其描述为把预训练模型的参数化记忆与外部非参数化记忆结合，用检索结果增强知识密集型生成任务。

3.1 RAG 基本链路

文档收集

- > 文档清洗
- > 文档切片
- > Embedding
- > 向量数据库存储
- > Query 优化
- > 检索召回
- > Rerank 精排
- > Prompt 构造
- > LLM 生成
- > 引用来源
- > 评估与监控

3.2 文档处理

RAG 中更准确的说法是“文档切片 / Chunking”，不是传统 NLP 里的“分词”。

文档处理需要关注：

- PDF / Word / Markdown / HTML 解析。
- 表格和图片 OCR。
- 目录、标题、页码、章节等元数据保留。
- 文档去重。
- 文档版本管理。
- 权限标签。
- 数据清洗和噪声过滤。

推荐为每个 chunk 保留元数据：

```
{
  "source": "policy.pdf",
  "page": 12,
  "section": "退款规则",
  "updated_at": "2026-05-24",
  "permission": "internal"
}
```

3.3 Chunking 策略

常见方式：

- Fixed-size Chunking：按固定字符数硬切，实现简单但容易切断语义。
- Recursive Character Splitting：按标题、段落、句子、标点递归切分，通用首选。
- Semantic Chunking：根据相邻句子 embedding 相似度决定切分点，语义更好但成本更高。
- Structure-aware Chunking：按 Markdown 标题、HTML DOM、PDF 目录、表格结构切分。
- Sliding Window Chunking：设置 overlap，避免边界信息丢失。
- Parent-Child Chunking：小 chunk 用于检索，大 chunk 或父文档用于生成。
- LLM-based Chunking：让模型判断最优切分点，效果可能更好但成本更高。

3.4 Embedding 与向量数据库

Embedding 用于把文本和 query 映射到同一个向量空间。

需要掌握：

- dense embedding
- sparse embedding
- multilingual embedding
- cosine similarity / dot product / L2 distance
- embedding 维度
- embedding 模型选型和评估

向量数据库选型：

- Chroma：轻量、本地友好、适合 demo 和小型项目。
- FAISS：高性能本地向量索引。
- Milvus / Zilliz：适合大规模分布式场景。
- Qdrant：过滤能力和工程体验较好。
- Weaviate：schema 和模块化能力较强。
- Elasticsearch / OpenSearch：适合 BM25 + 向量混合检索。

Chroma 的优势：

- 上手简单。
- 支持本地持久化。
- 适合学习和中小型知识库。
- 常见实现中结合 HNSW 近似最近邻索引和本地存储能力。

3.5 Query 优化

用户 query 往往口语化、不完整、缺少专业术语，因此需要 query 优化。

常见方法：

- Query Rewrite：把口语化问题改写成专业表达。
- Query Decomposition：复杂问题拆成多个子问题。
- Multi-query Retrieval：从多个角度改写 query，分别检索后合并。
- HyDE：先生成假设性答案文档，再用该文档向量检索真实文档。
- Step-back Prompting：把具体问题抽象成更上位的问题。
- NER / Keyword Extraction：抽取实体、时间、地点、编号、产品名等。
- Self-query Retriever：把自然语言转成结构化 metadata filter。
- Query Routing：判断应该去哪个知识库或工具查询。

3.6 BM25 与混合检索

BM25 是关键词精确匹配算法，解决 TF-IDF 中词频无限增长和长文档天然占优的问题。

核心思想：

- 词频增长要饱和，出现 10 次不应比出现 1 次强 10 倍。
- 文档长度要归一化，避免长文档因为词多更容易命中。
- IDF 表示词的区分度，越常见的词权重越低。

Embedding 和 BM25 的互补：

BM25：适合错误码、编号、专有名词、精确关键词
Embedding：适合同义表达、语义相似、模糊问题
Hybrid Search：两路召回后合并，再 rerank

常见融合：

- 加权分数融合。
- Reciprocal Rank Fusion, RRF。
- BM25 + vector top-k 合并后 rerank。

3.7 Rerank

Embedding 检索通常是 bi-encoder：

query 单独编码，document 单独编码，再计算相似度

Rerank 通常是 cross-encoder：

query 和 document 一起输入模型，直接输出相关性分数

典型流程：

```
召回 top 50 / top 100
-> rerank
-> 选出 top 5 / top 10
-> 放入 prompt
```

Rerank 更准确，但计算更慢，因此不适合对全量文档直接排序。

3.8 RAG 生成与引用

生成阶段要关注：

- 上下文去重。
- 按相关性或信息结构排序。
- 避免 lost-in-the-middle。
- 明确要求基于检索内容回答。
- 不足以回答时允许说不知道。
- 输出引用来源。
- 多文档冲突时说明冲突。
- 表格、时间线、政策类文档要保持结构。

3.9 RAG 评估

检索指标：

- Recall@K
- Precision@K
- Hit Rate
- MRR
- NDCG

生成指标：

- Faithfulness
- Answer Relevancy
- Context Precision
- Context Recall
- Citation Accuracy
- Hallucination Rate

端到端指标：

- 任务完成率。
- 用户满意度。
- 专家评分。
- 平均延迟。
- 平均成本。

4. Tool Calling 与外部能力接入

4.1 Tool 是什么

Tool 是外部函数或能力，用来让模型访问外部世界。

常见工具：

- 查询数据库。
- 读取文件。
- 搜索网页。
- 调用业务 API。
- 执行代码。
- 操作浏览器。
- 创建工单。
- 发送邮件。

模型本身不直接执行工具，而是输出工具调用意图，由 Agent Runtime 或应用后端执行。

4.2 Function Calling

Function Calling 是让模型以结构化方式请求调用工具的机制。

流程：

应用提供工具列表和参数 `schema`

- > 模型判断是否需要调用工具
- > 模型生成 `tool call`
- > 应用校验参数
- > 应用执行工具
- > 工具结果返回给模型
- > 模型生成最终答案

关键点：

- 工具描述要清晰。
- 参数 `schema` 要严格。
- 高风险工具要权限确认。
- 工具结果要结构化。
- 错误要可恢复。
- 所有调用要有 `trace` 和 `audit log`。

4.3 工具设计原则

好工具应该：

- 单一职责。
- 名称明确。
- 描述清楚。
- 参数尽量少。

- 返回结构化结果。
- 有超时限制。
- 有错误码和错误信息。
- 标记只读 / 写入 / 高风险。
- 支持幂等性。

不好的工具：

```
do_everything(input: string)
```

更好的工具：

```
get_order(order_id)
search_refund_policy(product_type)
cancel_order(order_id, reason)
```

4.4 工具治理

生产系统必须考虑：

- Tool Selection：工具太多时模型容易选错。
- Tool Routing：先缩小工具集合，再让模型选择。
- Permission：读、写、删除、支付等权限分级。
- Human-in-the-loop：高风险动作需要用户确认。
- Idempotency：避免重复扣款、重复下单、重复发邮件。
- Retry Policy：区分可重试和不可重试错误。
- Audit Log：记录工具名、参数、结果、耗时、用户授权。

5. MCP、Skills 与 CLI

5.1 MCP

MCP, Model Context Protocol, 是 Agent 和外部工具、资源、提示模板之间的标准化协议。

核心角色：

- Host：使用 MCP 的 AI 应用。
- Client：Host 内部连接某个 MCP Server 的客户端。
- Server：暴露 tools、resources、prompts 的服务。
- Tools：可执行动作。
- Resources：可读取上下文。
- Prompts：可复用提示模板。

典型流程：

```
Client 连接 Server
-> initialize
-> tools/list
-> 模型决定调用工具
-> tools/call
-> Server 返回结果
```

需要注意：

- MCP 解决的是工具和上下文服务的标准化接入。
- Function Calling 是模型侧的工具调用表达机制。
- MCP Server 暴露工具，Agent Runtime 把这些工具转成模型可用的工具描述。
- 当前 MCP 标准传输重点是 stdio 和 Streamable HTTP；旧的 HTTP+SSE 更多是兼容路径。

5.2 Skills

Skill 解决的是“如何按特定流程完成任务”。

一个 Skill 通常包含：

- SKILL.md：核心说明。
- scripts：可执行脚本。
- references：参考资料。
- templates：模板文件。
- assets：辅助资源。

Skill 的关键机制是渐进式披露：

```
启动时只加载 skill 名称和描述
-> 任务相关时读取 SKILL.md
-> 需要时再读取参考资料或执行脚本
```

Skill 适合封装：

- 代码审查流程。
- PDF 处理流程。
- 数据分析流程。
- 简历评估流程。
- 安全审计流程。
- 发布检查流程。

5.3 CLI

CLI 在 Coding Agent 中非常重要。

优势：

- token 成本低。
- 表达能力强。
- 可以组合多个程序。

- 适合搜索、测试、构建、git、文件操作。
- 工程生态成熟。

风险：

- 命令可能不安全。
- 模型可能生成错误命令。
- 输出可能过长。
- 需要权限、沙箱和审计。

建议理解：

MCP：结构化、标准化、可治理
CLI：灵活、高效、工程覆盖广
Skills：把经验流程沉淀下来

6. Agent 架构层

6.1 Agent 是什么

Agent 可以理解为：

LLM + 状态 + 规划 + 工具 + 记忆 + 执行循环 + 反馈机制

真正的 Agent 不只是调用一次工具，而是具备：

- 目标理解。
- 任务拆解。
- 工具选择。
- 多步执行。
- 观察反馈。
- 状态更新。
- 错误恢复。
- 终止判断。

6.2 ReAct

ReAct 的核心是推理和行动交替进行。

Thought -> Action -> Observation -> Thought -> Action -> Observation -> Final Answer

实现方式：

- 系统提示词定义格式。
- 模型生成 action。
- Runtime 解析工具名和参数。
- 执行工具。

- observation 加入上下文。
- 继续循环直到 final answer 或达到最大步数。

优点：

- 灵活。
- 适合探索型任务。
- 可以根据工具反馈纠错。

缺点：

- 多轮调用，成本高。
- 长链路容易积累错误。
- 工具描述不清会导致误调用。
- 上下文容易膨胀。

6.3 Plan-and-Execute

流程：

```
Planner 生成计划
-> Executor 执行每一步
-> Verifier 检查结果
-> 必要时 Replan
```

适合：

- 目标明确。
- 步骤相对稳定。
- 可以提前规划的任务。

缺点：

- 原始版本缺少动态纠错。
- 中间步骤失败时容易计划失效。

生产中通常要加入：

- step-level verification。
- dynamic replanning。
- checkpoint。
- rollback。

6.4 Reflexion

Reflexion 通过“执行、评估、反思、记忆”形成迭代改进。

组件：

- Actor：执行任务。
- Evaluator：评估结果。
- Reflector：总结失败原因和改进建议。

- Memory: 保存反思, 供下一轮使用。

适合:

- 代码生成。
- 多次尝试型任务。
- 问答纠错。
- 复杂决策任务。

6.5 多智能体

多智能体适合复杂任务协作。

常见角色:

- Planner
- Researcher
- Executor
- Reviewer
- Critic
- Memory Manager
- Tool Router
- Security Guard

优点:

- 专业化分工。
- 上下文隔离。
- 可以并行执行。
- 易于扩展。

缺点:

- token 成本高。
- 延迟更大。
- 协调复杂。
- 结果合并和冲突处理困难。

判断是否需要多 Agent:

- 任务是否可以自然拆分?
- 是否需要不同专业角色?
- 是否需要独立评审?
- 单 Agent 是否已经因为工具太多或上下文太长而不稳定?

6.6 A2A

A2A, Agent-to-Agent Protocol, 解决 Agent 与 Agent 之间如何互操作。

核心概念：

- Agent Card：被调用 Agent 的能力描述。
- Message：Agent 之间交换的信息。
- Task：需要完成的任务。
- Artifact：Agent 产出的结果。
- Part：Message 或 Artifact 中的最小内容单元。

可以这样区分：

MCP: Agent 调工具
A2A: Agent 调 Agent
Skill: Agent 学流程

7. Memory 与 Context Engineering

7.1 短期记忆

短期记忆就是当前会话上下文，包括：

- 用户输入。
- 模型回复。
- 工具调用。
- 工具结果。
- 当前计划。
- 当前任务状态。

问题：

- token 成本高。
- 噪声越来越多。
- 工具结果可能撑爆上下文。
- 旧信息可能干扰新任务。

7.2 长期记忆

长期记忆是跨会话持久化信息。

可以存储：

- 用户偏好。
- 项目规范。
- 历史任务总结。
- 常用命令。
- 失败经验。

- 业务知识。

存储方式：

- Markdown 文件。
- SQLite / Postgres。
- 向量数据库。
- Knowledge Graph。
- Object Storage。
- Event Log。

7.3 上下文工程

可以总结为四件事：

write: 哪些信息写入上下文或记忆
select: 当前任务选择哪些信息
compress: 如何压缩历史和工具结果
isolate: 如何隔离子任务和子 **Agent** 上下文

7.4 上下文压缩

常见策略：

- 截断。
- 摘要。
- 结构化摘要。
- 工具结果只保留 preview。
- 大文件 offload 到外部，只保留 path / id。
- 每个子任务结束后生成 checkpoint。
- 定期生成 session summary。

摘要必须保留：

- 用户目标。
- 明确约束。
- 当前进展。
- 已完成事项。
- 待办事项。
- 关键文件路径。
- 重要错误和修复方式。
- 当前决策依据。

可以丢弃：

- 重复寒暄。
- 无关搜索结果。
- 过时计划。

- 已解决的中间错误。
- 大段重复工具输出。

7.5 记忆污染治理

长期记忆不是越多越好。

需要治理：

- 写入门槛。
- 来源记录。
- 时间戳。
- 置信度。
- 用户可见和可编辑。
- 冲突检测。
- 过期清理。
- 定期合并和整理。

8. Agent 评估体系

Agent 评估不能只看最终答案是否“看起来不错”。因为 Agent 是一个多步执行系统，可能最终答案正确，但中间工具调用混乱、成本极高、证据不足；也可能最终答案格式正确，但任务实际没有完成。

因此 Agent 评估应分层设计。

8.1 第一层：任务完成层

问题：

用户给的任务最终是否完成？

指标：

- Task Success Rate：任务完成率。
- Failure Rate：失败率。
- Partial Success Rate：部分完成率。
- Error Rate：异常错误率。
- Retry Success Rate：失败后重试成功率。
- Human Intervention Rate：需要人工介入比例。

示例：

如果任务是“生成简历评估报告并给出分数”，则检查：

- 是否成功读取简历。
- 是否成功调用评估标准。
- 是否生成完整报告。
- 是否输出总分和各项分数。
- 是否出现中间异常导致任务失败。

这一层是最低要求，但不代表质量好。

8.2 第二层：结果质量层

问题：

任务完成了，但结果是否准确、有用、符合专家判断？

指标：

- Accuracy：答案或判断是否正确。
- Expert Agreement：与人类专家评分的一致性。
- Score Deviation：模型评分与专家评分的平均误差。
- Rubric Compliance：是否遵守评分规则。
- User Satisfaction：用户满意度。
- Completeness：输出是否完整覆盖任务要求。

评估方式：

- 构建黄金数据集。
- 邀请专家对同一输入进行评分。
- 比较 Agent 输出和专家输出的误差。
- 对主观任务使用多专家平均分或仲裁分。

示例：

简历评估任务中，可以让人类专家对候选人从“教育背景、项目经历、技术能力、岗位匹配度、表达质量”等维度打分，然后比较 Agent 打分和专家打分的差异。

8.3 第三层：证据与忠实度层

问题：

Agent 的结论是否有证据支持？有没有编造依据？

指标：

- Evidence Coverage：关键结论是否有证据覆盖。
- Missing Evidence Rate：证据缺失率。
- Unsupported Claim Rate：无依据断言比例。
- Citation Accuracy：引用是否指向正确来源。
- Faithfulness：输出是否忠实于输入材料和检索内容。
- Groundedness：答案是否建立在可验证证据上。

示例：

简历评估任务中，每项评分都应该给出依据：

技术能力：8/10

依据：候选人具备 Java、Spring Boot、MySQL 项目经验，并在 XX 项目中负责核心接口设计。

如果模型给出“候选人具备大规模分布式系统经验”，但简历里没有证据，这就是 unsupported claim。

8.4 第四层：规划与工具调用层

问题：

Agent 完成任务的过程是否合理？

指标：

- Tool Call Accuracy：是否调用了正确工具。
- Tool Call Precision：调用的工具中有多少是必要的。
- Tool Call Recall：应该调用的工具是否都调用了。
- Parameter Accuracy：工具参数是否正确。
- Tool Sequence Accuracy：工具调用顺序是否合理。
- Redundant Tool Call Rate：重复或无效工具调用比例。
- Missing Tool Call Rate：遗漏必要工具的比例。
- Plan Correctness：任务规划是否合理。
- Step Completion Rate：计划步骤完成率。

示例：

简历评估 Agent 的合理工具链可能是：

读取简历 -> 解析简历结构 -> 读取评分标准 -> 匹配岗位要求 -> 生成分项评分 -> 生成报告

如果 Agent 没有读取评分标准就直接打分，说明工具和规划层有问题；如果反复读取同一个文件 5 次，说明效率和状态管理有问题。

8.5 第五层：成本与效率层

问题：

Agent 是否足够快、足够便宜，能否投入真实应用？

指标：

- Average Latency：平均完成时间。
- P95 / P99 Latency：尾延迟。
- Token Cost per Task：单任务 token 成本。
- Tool Cost per Task：外部 API 或工具成本。
- Total Cost per Successful Task：每个成功任务的平均成本。
- Tokens per Completed Step：每完成一步消耗多少 token。
- Retry Count：平均重试次数。
- Context Growth Rate：上下文增长速度。

这层非常关键，因为生产系统不只看“能不能做”，还要看“做一次多少钱、用户等多久、失败后浪费多少”。

8.6 第六层：鲁棒性与安全层

问题：

Agent 在异常、攻击、边界场景下是否稳定？

指标：

- Prompt Injection Resistance：抗提示词注入能力。
- Permission Violation Rate：越权操作比例。
- Unsafe Action Rate：危险动作比例。
- Data Leakage Rate：敏感信息泄露比例。
- Format Stability：输出格式稳定性。
- Recovery Rate：工具失败后的恢复率。
- Robustness under Noisy Input：噪声输入下的稳定性。

测试方式：

- 对抗样本。
- 工具失败模拟。
- 网络超时模拟。
- 空输入、脏数据、重复数据测试。
- 权限边界测试。
- 恶意文档注入测试。

8.7 第七层：线上监控与回归评估

Agent 上线后要持续监控。

需要记录：

- 每次任务输入。
- 模型输出。
- 工具调用轨迹。
- 工具参数。
- 工具返回。
- token 消耗。
- 延迟。
- 用户反馈。
- 错误堆栈。
- 人工接管情况。

离线评估：

- 固定测试集。
- Golden Dataset。
- 版本回归测试。
- Prompt A/B test。

- 模型 A/B test。
- 工具描述变更回归。

在线评估：

- 用户反馈。
- 采样人工审核。
- 线上 trace 评分。
- 异常任务聚类。
- 成本和延迟监控。

8.8 Agent 评估总表

层级	评估问题	核心指标
任务完成层	是否完成用户目标	Task Success Rate, Failure Rate
结果质量层	结果是否准确、有用	Accuracy, Expert Agreement, Score Deviation
证据层	结论是否有依据	Evidence Coverage, Faithfulness, Citation Accuracy
规划工具层	中间过程是否合理	Tool Call Accuracy, Parameter Accuracy, Plan Correctness
成本效率层	是否快且便宜	Latency, Token Cost, Cost per Success
安全鲁棒层	是否安全稳定	Injection Resistance, Permission Violation, Recovery Rate
线上监控层	是否持续可控	Trace Review, Regression Eval, User Feedback

8.9 一个简历评估 Agent 的评估设计示例

任务：

输入简历和岗位 JD，输出候选人评估报告、分项评分、总分和录用建议。

评估集：

- 100 份简历。
- 10 类岗位 JD。
- 每份简历由 2-3 名人类专家评分。
- 保留专家评分均值和解释作为参考答案。

指标：

- 报告生成成功率。
- 总分与专家均分的 MAE。
- 分项评分与专家评分的 MAE。

- 每项评分证据缺失率。
- 无依据断言比例。
- 是否正确读取简历、JD、评分标准。
- 工具调用次数是否在合理范围。
- 单次任务平均 token 成本。
- 平均生成时间。
- 用户满意度。

评估结论示例：

任务成功率：96%
总分 MAE：0.8 / 10
证据缺失率：7%
工具调用准确率：92%
平均成本：\$0.06 / task
平均耗时：18s
主要问题：对项目经历深度的评分偏乐观，且在低质量简历中证据引用不足。

9. Agent Runtime 与 Harness Engineering

Harness Engineering 是围绕 LLM 构建的一套运行时系统，用来约束、调度、观察、验证和纠正模型行为。

9.1 Agent Loop

Agent Loop 通常维护：

- messages
- tools
- memory
- plan
- current step
- observations
- token usage
- retry count
- max iterations
- stop reason

9.2 Tool Registry

每个工具应记录：

- name
- description
- input schema
- output schema
- read/write 标记

- risk level
- timeout
- retry policy
- parallelizable
- reversible
- permission rule

9.3 权限系统

建议分层：

模型层：能不能看到或请求这个工具
应用层：参数是否合法
权限层：用户是否授权
沙箱层：物理上能不能执行
审计层：执行后是否记录

9.4 沙箱

尤其是 CLI 和 Coding Agent，必须考虑：

- 文件系统边界。
- 网络边界。
- 命令白名单 / 黑名单。
- 写操作审批。
- destructive command 拦截。
- secret 防泄露。
- patch review。
- test before commit。

9.5 可观测性

生产 Agent 必须能回答：

- 为什么调用这个工具？
- 调用了什么工具？
- 参数是什么？
- 结果是什么？
- 哪一步失败？
- token 花在哪里？
- prompt 或工具描述变更是否造成回归？

需要：

- tracing
- logging
- span

- cost tracking
- latency tracking
- tool call audit
- eval dashboard

10. 推荐项目路线

项目 1：结构化输出信息抽取器

目标：

- 掌握 Prompt。
- 掌握 JSON Schema。
- 掌握输出校验。
- 掌握失败重试。

功能：

- 输入简历 / 合同 / 商品描述。
- 输出结构化 JSON。
- schema 校验。
- 错误重试。
- 置信度字段。

项目 2：本地知识库 RAG 系统

目标：

- 掌握完整 RAG 链路。

功能：

- 文档上传。
- Markdown / PDF 解析。
- chunking。
- embedding。
- Chroma / Qdrant。
- BM25 + vector hybrid。
- rerank。
- answer with citations。
- RAG 评估。

项目 3：工具调用型客服 Agent

目标：

- 掌握 Function Calling 和 Tool Runtime。

功能：

- 查订单。
- 查物流。
- 查退款政策。
- 创建售后工单。
- 高风险操作二次确认。
- tool call trace。
- 参数校验。

项目 4：ReAct Coding Agent Mini 版

目标：

- 掌握 Agent Loop。

功能：

- read file。
- search code。
- run shell。
- apply patch。
- run tests。
- max iteration。
- stop condition。
- error retry。
- task summary。

项目 5：类 Claude Code / OpenClaw Agent Runtime

目标：

- 掌握生产级 Agent 架构。

功能：

- Tool Registry。
- MCP Client。
- Skill Loader。
- Memory.md。
- Session Summary。
- Context Compression。
- Permission System。
- Sandbox。
- Multi-Agent Task。
- Evaluator。
- Trace Dashboard。

11. 最终路线总图

LLM 基础

- > Prompt 与结构化输出
- > RAG 知识增强
- > Tool Calling 外部能力
- > MCP / Skills / CLI 工具生态
- > ReAct / Plan-and-Execute / Reflexion
- > Memory / Context Engineering
- > Multi-Agent / A2A
- > Agent Eval
- > Harness Engineering
- > 生产级 Agent Runtime

12. 学习重点排序

如果时间有限，建议按优先级学习：

1. LLM 基础和生成机制。
2. Prompt 与结构化输出。
3. RAG 全链路。
4. Function Calling 和工具设计。
5. ReAct Agent Loop。
6. Agent 评估体系。
7. Memory 和上下文工程。
8. MCP、Skills、CLI。
9. 权限、沙箱、可观测性。
10. 多 Agent 和 A2A。